

Neural Language Models

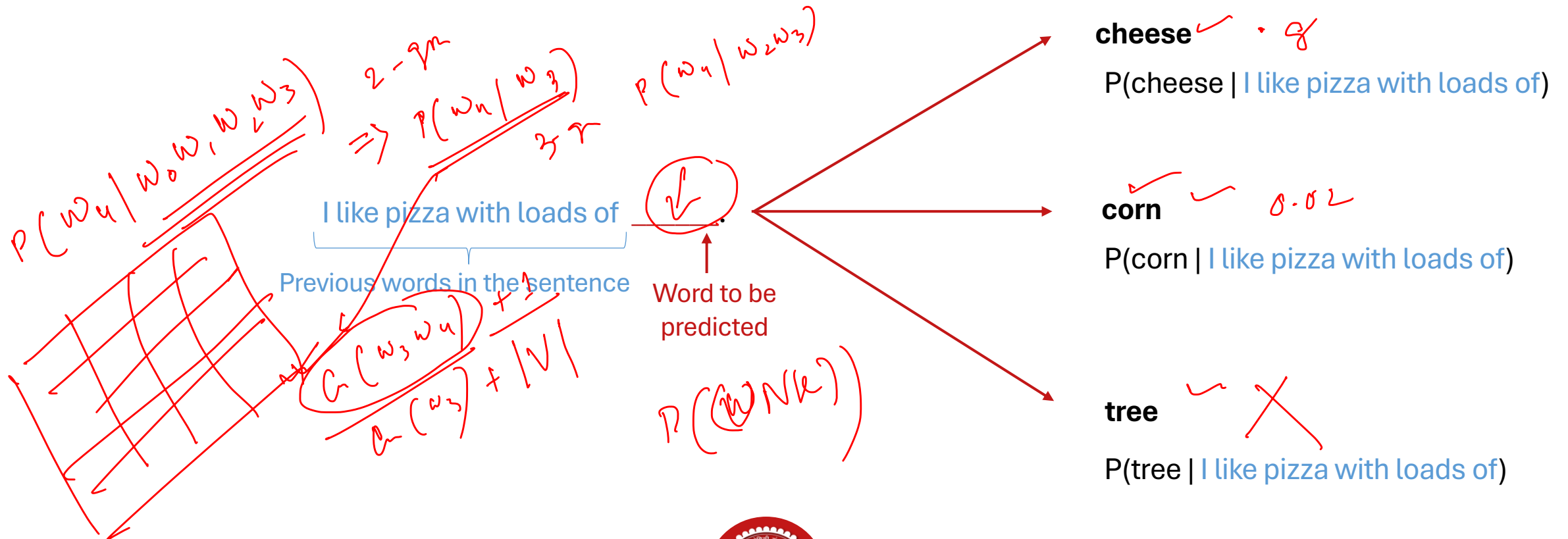


Tanmoy Chakraborty
Associate Professor, IIT Delhi
<https://tanmoychak.com/>

Recall: Language Modeling

$$P(w_1, w_2, \dots, w_n) = P(w_1) P(w_2 | w_1) \dots P(w_n | w_1, \dots, w_{n-1})$$

- Language Modeling** is the task of predicting what word comes next



Recall: Language Modeling

- You can also think of a Language Model as a system that assigns a probability to a piece of text.
- For example, if we have some text $x^{(1)}, \dots, x^{(T)}$, then the probability of this text (according to the Language Model) is:

$$\begin{aligned} P(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}) &= P(\mathbf{x}^{(1)}) \times P(\mathbf{x}^{(2)} | \mathbf{x}^{(1)}) \times \dots \times P(\mathbf{x}^{(T)} | \mathbf{x}^{(T-1)}, \dots, \mathbf{x}^{(1)}) \\ &= \prod_{t=1}^T \underbrace{P(\mathbf{x}^{(t)} | \mathbf{x}^{(t-1)}, \dots, \mathbf{x}^{(1)})} \end{aligned}$$

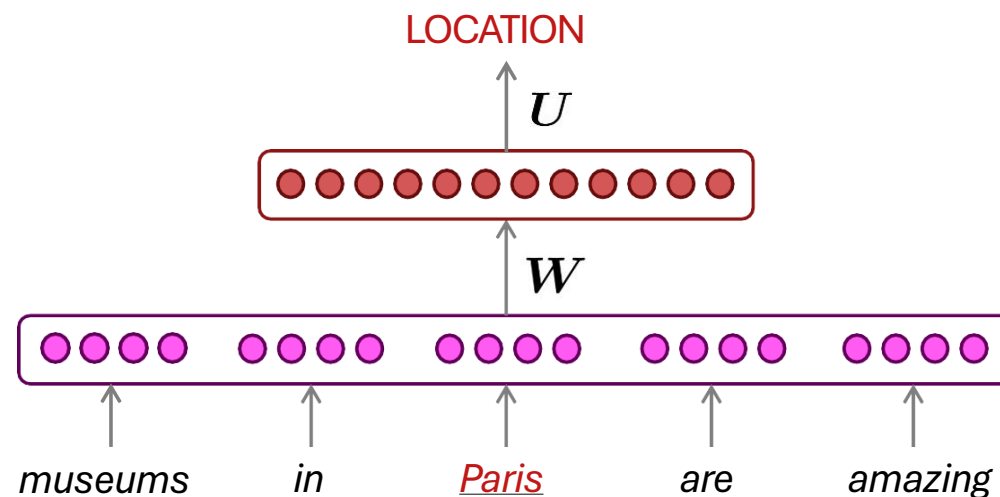
This is what our LM provides



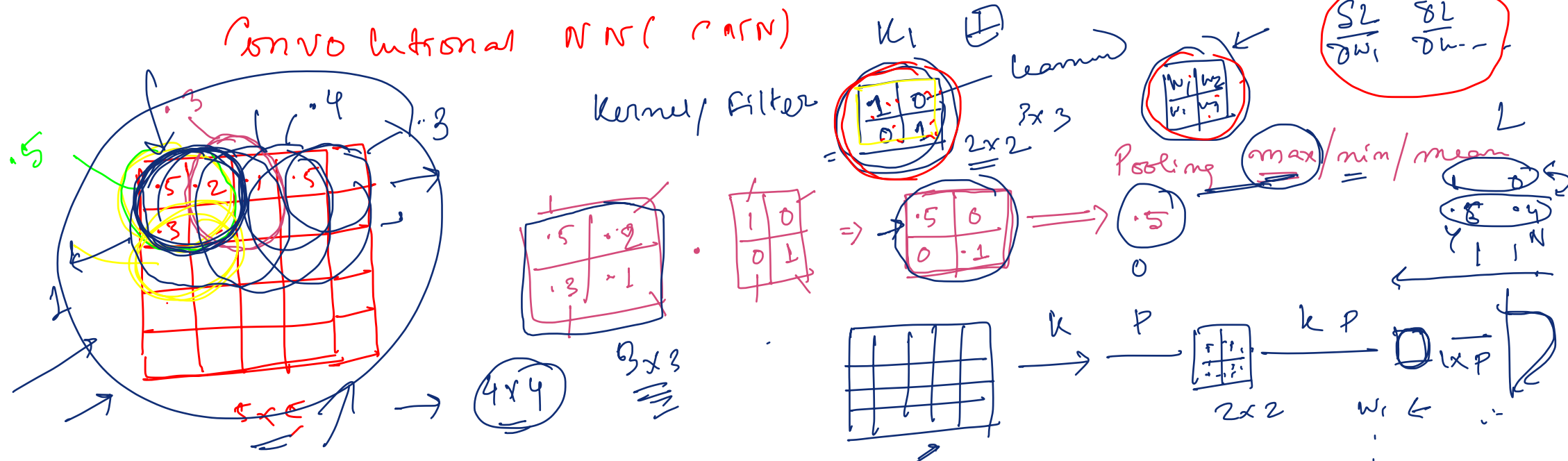
How to Build a *Neural* Language Model?

- Recall the Language Modeling task:
 - **Input:** sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(t)}$
 - **Output:** probability distribution of the next word $P(x^{(t+1)} | x^{(t)}, \dots, x^{(1)})$
- How about a window-based neural model?

Example: NER Task



A Fixed-window Neural Language Model



as the proctor started the clock the students opened their

discard

fixed window



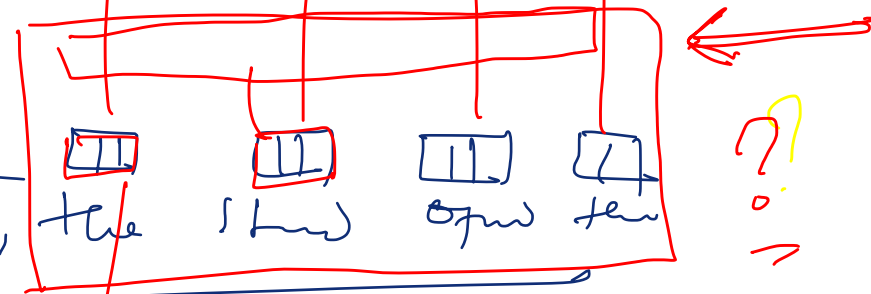
Min Embedding
Matrix

$U_{400 \times 101}$



$V = 400$

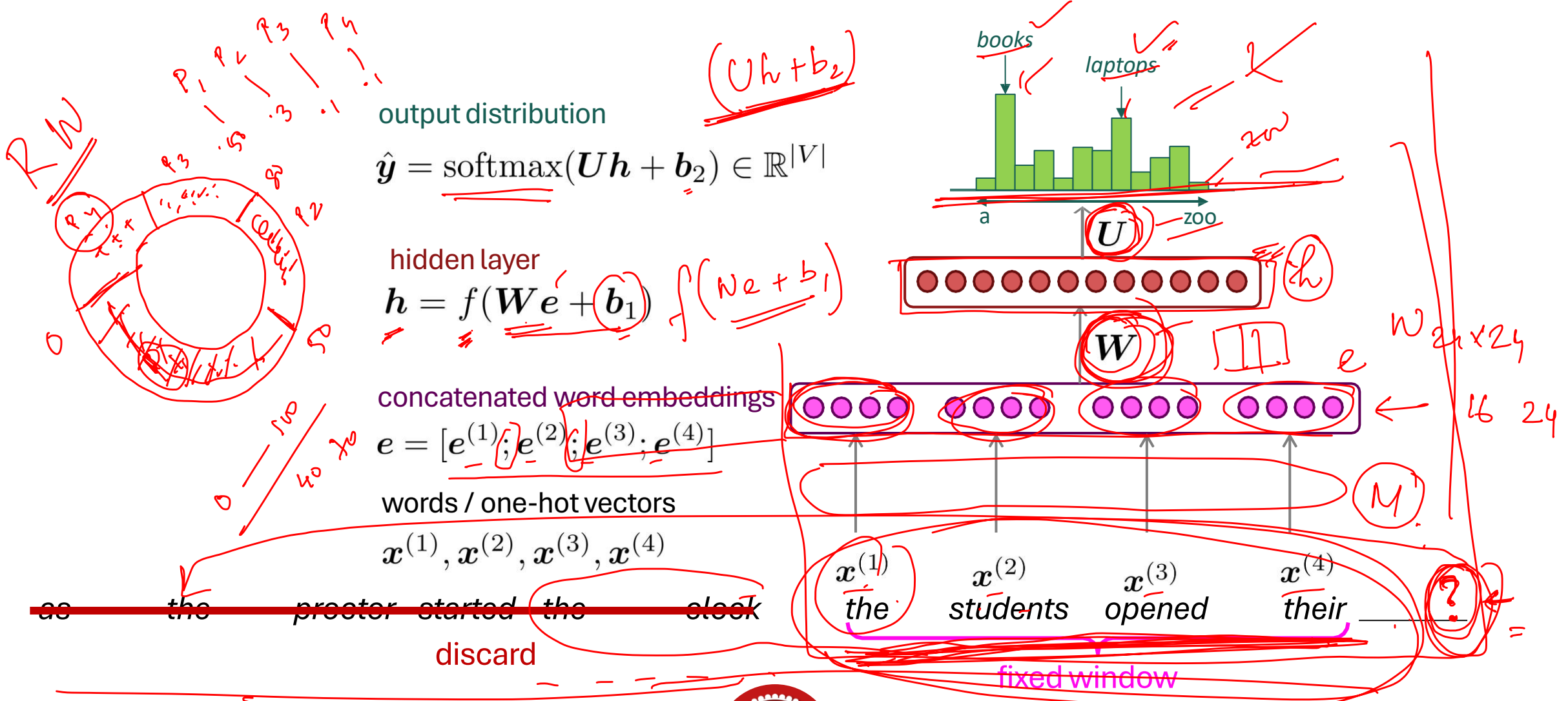
W



~~as the processor shows the clock, the slow operation~~

100

A Fixed-window Neural Language Model



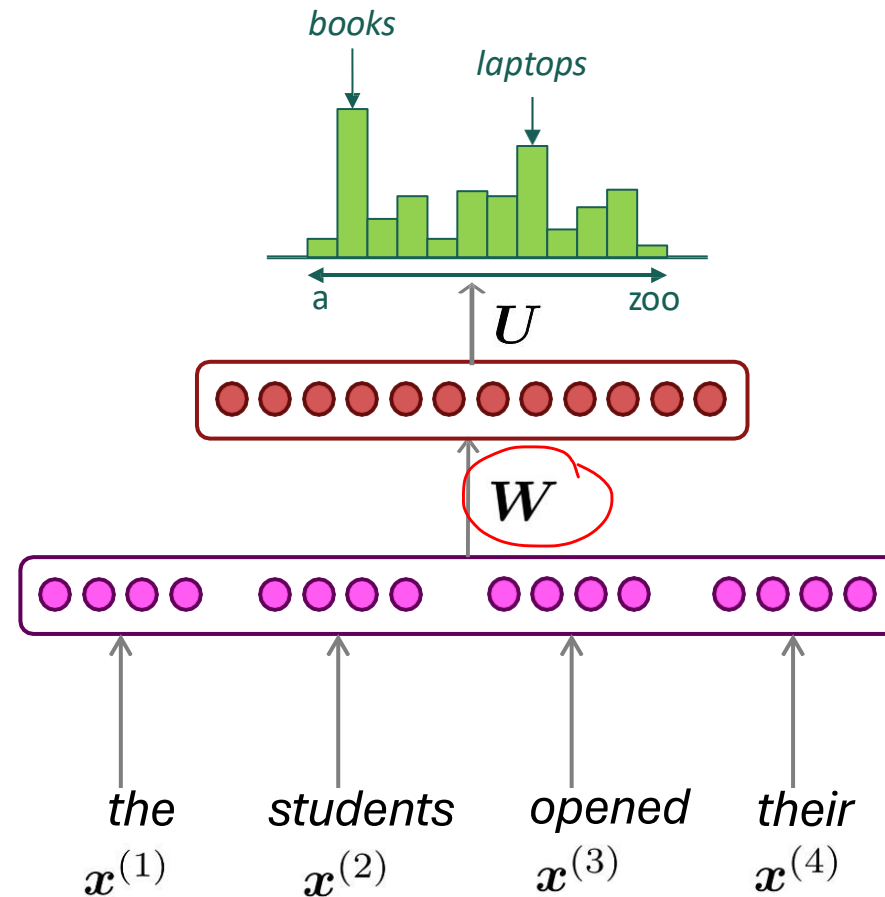
A Fixed-window Neural Language Model

Improvements over n -gram LM:

- No sparsity problem
- Don't need to store all observed n -grams

Remaining problems:

- Fixed window is **too small**
- Enlarging window enlarges W
- $x^{(1)}$ and $x^{(2)}$ are multiplied by completely different weights in W .
No symmetry in how the inputs are processed.

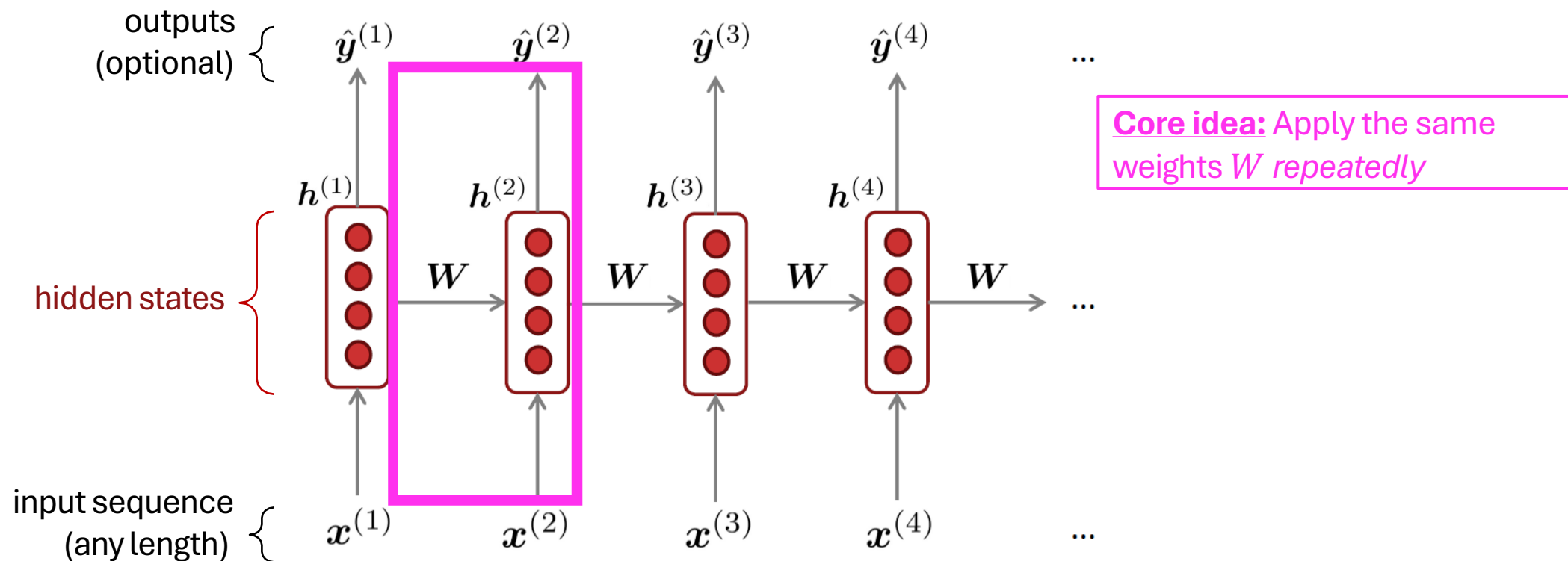


Approximately: Y. Bengio, et al.
(2000/2003): A Neural Probabilistic
Language Model

We need a neural
architecture that can
process **any length**
input



Recurrent Neural Networks (RNN)

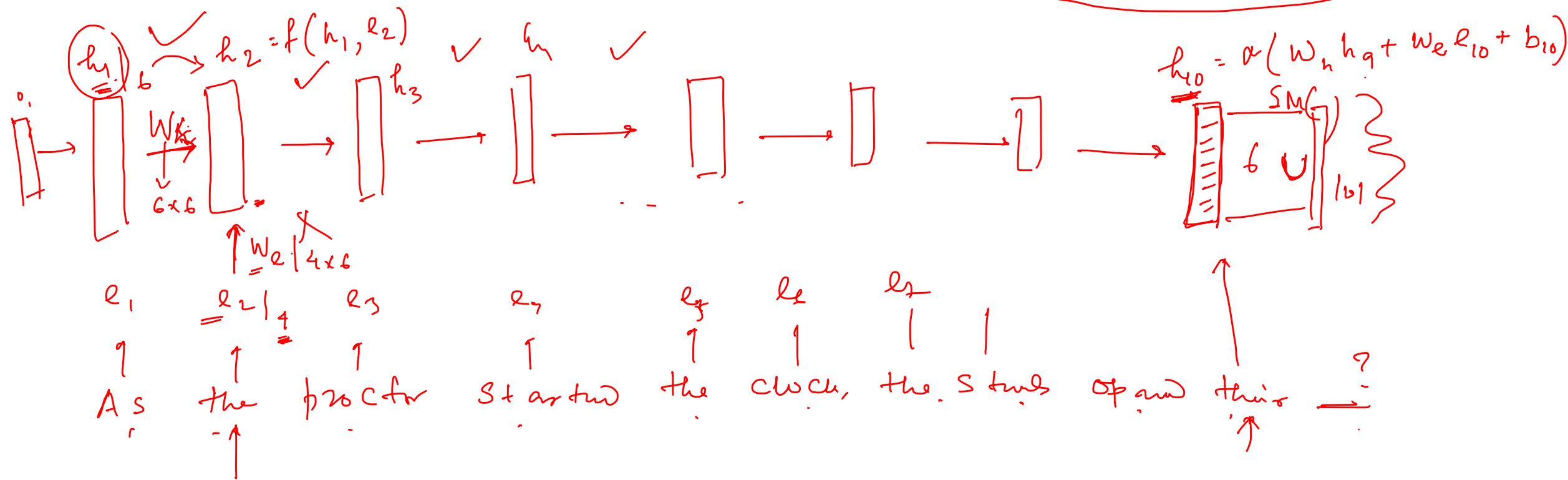
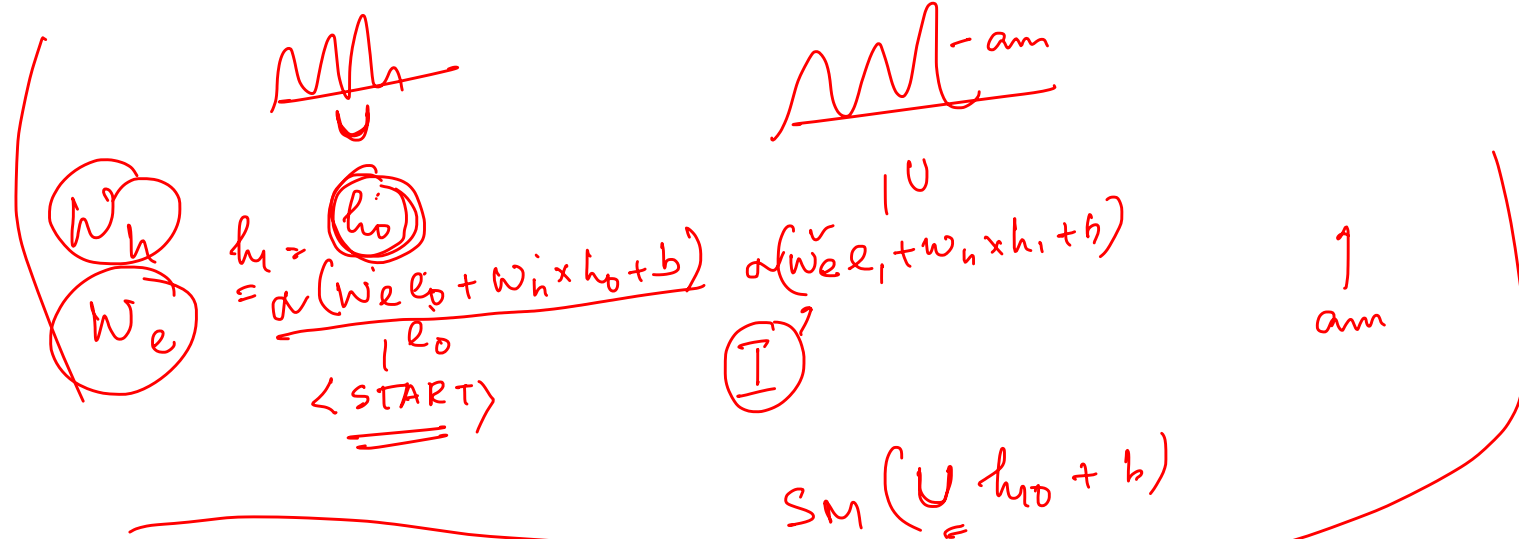


$$h_3 = \alpha(w_e l_3 + w_h h_2 + b_3)$$

$$h_2 = \alpha(w_e l_2 + w_h h_1 + b_2)$$

$$\square + \square + \square$$

$$h_2 = \alpha(w_h h_1 + w_e l_2 + b_2)$$



A Simple RNN Language Model

output distribution

$$\hat{y}^{(t)} = \text{softmax}(U h^{(t)} + b_2) \in \mathbb{R}^{|V|}$$

hidden states

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

$h^{(0)}$ is the initial hidden state

word embeddings

$$e^{(t)} = E x^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$

Note: this input sequence could be much longer now!

N_h W_e

$$\hat{y}^{(4)} = P(x^{(5)} | \text{the students opened their})$$

books

laptops

a

zoo

$h^{(0)}$

$h^{(1)}$

$h^{(2)}$

$h^{(3)}$

$h^{(4)}$

W_h

W_h

W_h

W_h

W_e

W_e

W_e

W_e

$e^{(1)}$

$e^{(2)}$

$e^{(3)}$

$e^{(4)}$

E

E

E

E

the

students

opened

their

$x^{(1)}$

$x^{(2)}$

$x^{(3)}$

$x^{(4)}$

U

Embedding projection layer



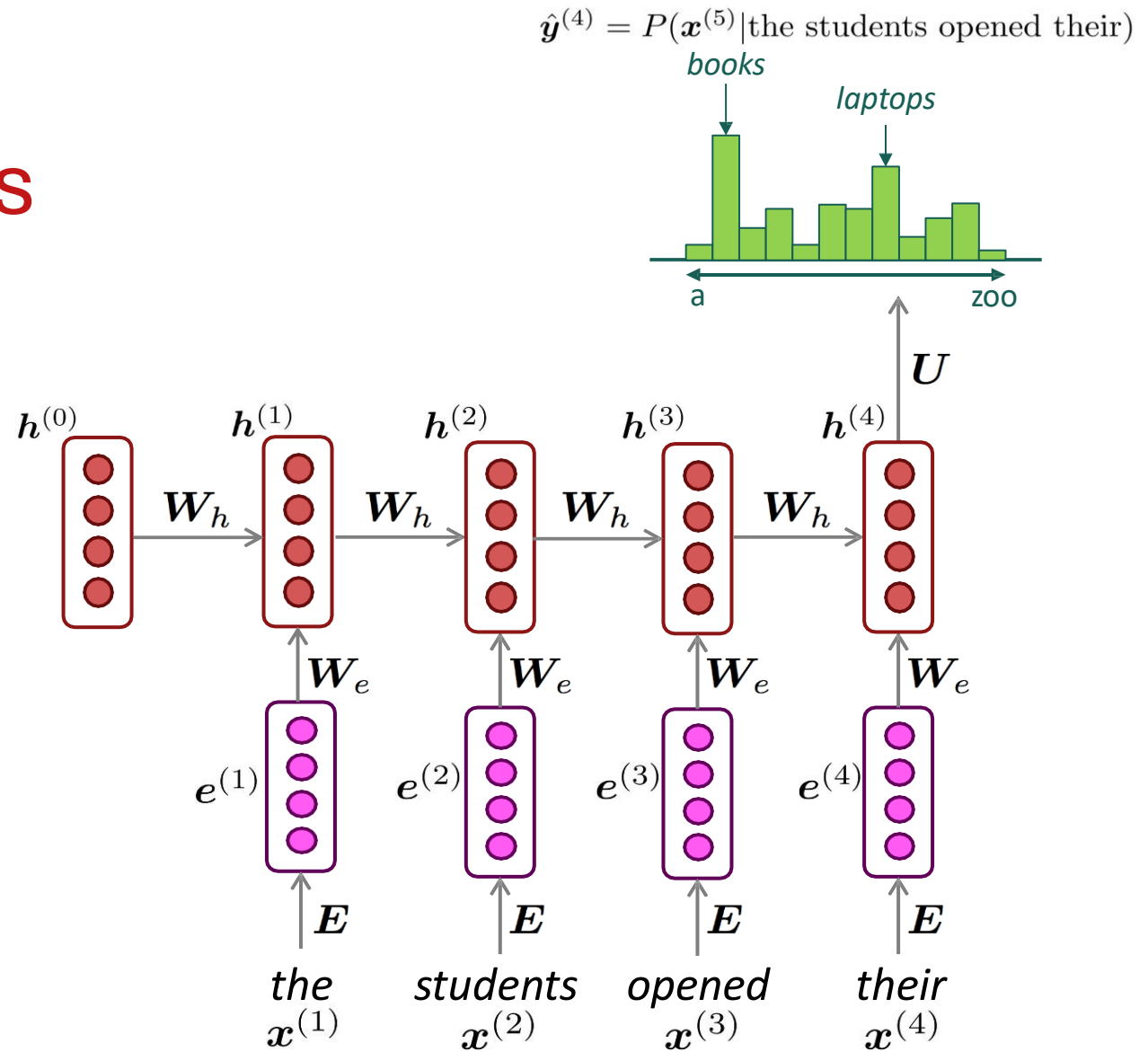
RNN Language Models

RNN Advantages:

- Can process **any length** input
- Computation for step t can (in theory) use information from **many steps back**
- **Model size doesn't increase** for longer input context
- Same weights applied on every timestep, so there is **symmetry** in how inputs are processed.

RNN Disadvantages:

- Recurrent computation is **slow**
- In practice, difficult to access information from **many steps back**



Training an RNN Language Model

Training an RNN Language Model

- Get a **big corpus of text** which is a sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(T)}$
- Feed into RNN-LM; compute output distribution $\hat{y}^{(t)}$ **for every step t** .
 - i.e., predict probability distribution of every word, given words so far
- **Loss function** on step t is **cross-entropy** between predicted probability distribution $\hat{y}^{(t)}$, and the true next word $y^{(t)}$ (one-hot for $x^{(t+1)}$):

$$J^{(t)}(\theta) = CE(\mathbf{y}^{(t)}, \hat{\mathbf{y}}^{(t)}) = - \sum_{w \in V} \mathbf{y}_w^{(t)} \log \hat{\mathbf{y}}_w^{(t)} = - \log \hat{\mathbf{y}}_{\mathbf{x}_{t+1}}^{(t)}$$

- Average this to get **overall loss** for entire training set:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^T - \log \hat{\mathbf{y}}_{\mathbf{x}_{t+1}}^{(t)}$$



$$J' = CE(\hat{y}, y)$$

“Teacher forcing”

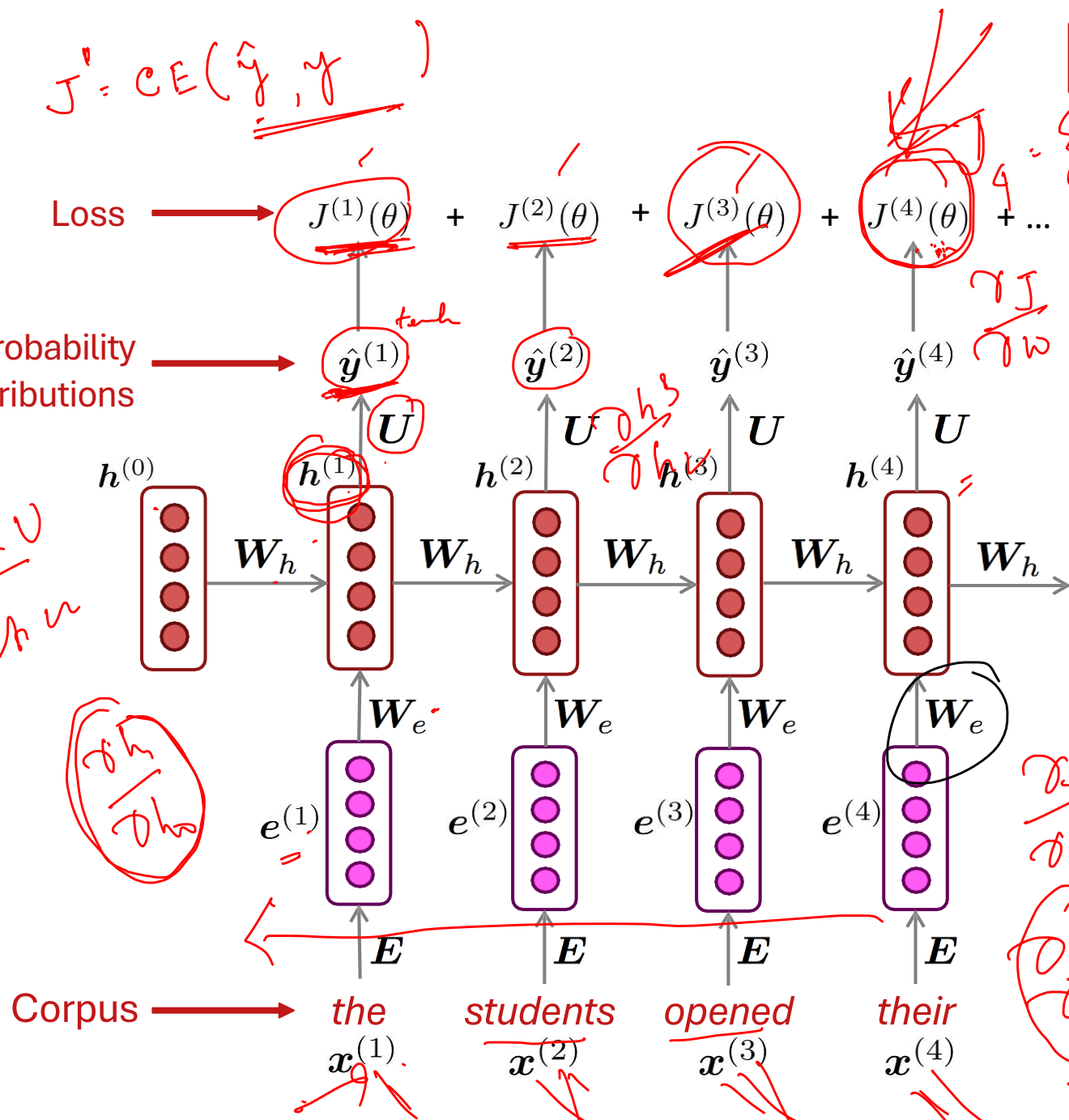
Loss $\longrightarrow J^{(1)}(\theta) + J^{(2)}(\theta) + J^{(3)}(\theta) + J^{(4)}(\theta) + \dots = J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta)$

Predicted probability distributions

W_n^F
 W_e^F

LSTM/GRU
w/ Attention

①
②



BTT

$\frac{\partial J}{\partial h_3} = \frac{\partial J}{\partial h_4} = \frac{\partial J}{\partial h_2} = \frac{\partial J}{\partial h_1} = \frac{\partial J}{\partial y}$

exams ...

chi ...

Training a RNN Language Model

- However: Computing loss and gradients across **entire corpus** $x^{(1)}, x^{(2)}, \dots, x^{(T)}$ at once is **too expensive** (memory-wise)!

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta)$$

- In practice, consider $x^{(1)}, x^{(2)}, \dots, x^{(T)}$ as a **sentence** (or a **document**)
- Recall: **Stochastic Gradient Descent** allows us to compute loss and gradients for small chunk of data, and update.
- Compute loss $J(\theta)$ for a sentence (actually, a batch of sentences), compute gradients and update weights. Repeat on a new batch of sentences.



Neural Language Models

Large Language Models: Introduction and Recent Advances

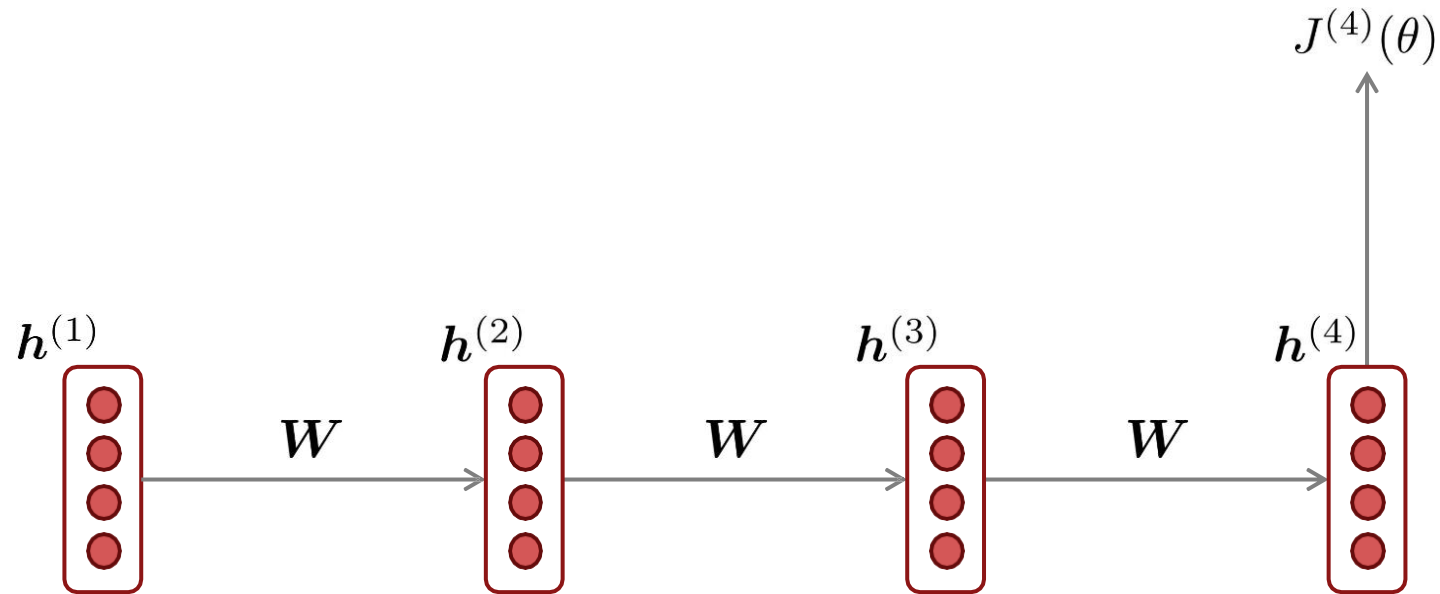
ELL881 · AIL821



Tanmoy Chakraborty
Associate Professor, IIT Delhi
<https://tanmoychak.com/>

Problems with RNNs

Vanishing and Exploding Gradients



Effect of Vanishing Gradient on RNN-LM

- **LM task:** *When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her _____*
- To learn from this training example, the RNN-LM needs to **model the dependency** between “*tickets*” on the 7th step and the target word “*tickets*” at the end.
- But if the gradient is small, the model **can't learn this dependency**
 - So, the model is **unable to predict similar long-distance dependencies** at test time



How to Fix the Vanishing Gradient Problem?

- The main problem is that *it's too difficult for the RNN to learn to preserve information over many timesteps.*
- In a vanilla RNN, the hidden state is constantly being rewritten

$$\mathbf{h}^{(t)} = \sigma \left(\mathbf{W}_h \mathbf{h}^{(t-1)} + \mathbf{W}_x \mathbf{x}^{(t)} + \mathbf{b} \right)$$

- How about an RNN with separate **memory** which is added to?
 - LSTMs
- And then: Creating more direct and linear pass-through connections in model
 - Attention, residual connections, etc.



And.

LSTMs & GRUs

Transf.



Long Short-Term Memory (LSTM)

- A type of RNN proposed by Hochreiter and Schmidhuber in 1997 as a solution to the vanishing gradients problem.
- On step t , there is a hidden state $h^{(t)}$ and a cell state $c^{(t)}$
 - Both are vectors length n
 - The cell stores long-term information
 - The LSTM can erase, write and read information from the cell
- The selection of which information is erased/written/read is controlled by three corresponding gates (gates are calculated things whose values are probabilities)
 - The gates are also vectors length n
 - On each timestep, each element of the gates can be open (1), closed (0), or somewhere in-between.
 - The gates are dynamic: their value is computed based on the current context



LSTM

We have a sequence of inputs $x^{(t)}$, and we will compute a sequence of hidden states $h^{(t)}$ and cell states $c^{(t)}$. On timestep t :

✓ **Forget gate:** controls what is kept vs forgotten, from previous cell state

✓ **Input gate:** controls what parts of the new cell content are written to cell

✓ **Output gate:** controls what parts of cell are output to hidden state

New cell content: this is the new content to be written to the cell

Cell state: erase (“forget”) some content from last cell state, and write (“input”) some new cell content

Hidden state: read (“output”) some content from the cell

Sigmoid function: all gate values are between 0 and 1

$$f^{(t)} = \sigma(W_f h^{(t-1)} + U_f x^{(t)} + b_f)$$

$$i^{(t)} = \sigma(W_i h^{(t-1)} + U_i x^{(t)} + b_i)$$

$$o^{(t)} = \sigma(W_o h^{(t-1)} + U_o x^{(t)} + b_o)$$

$$\tilde{c}^{(t)} = \tanh(W_c h^{(t-1)} + U_c x^{(t)} + b_c)$$

$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)}$$

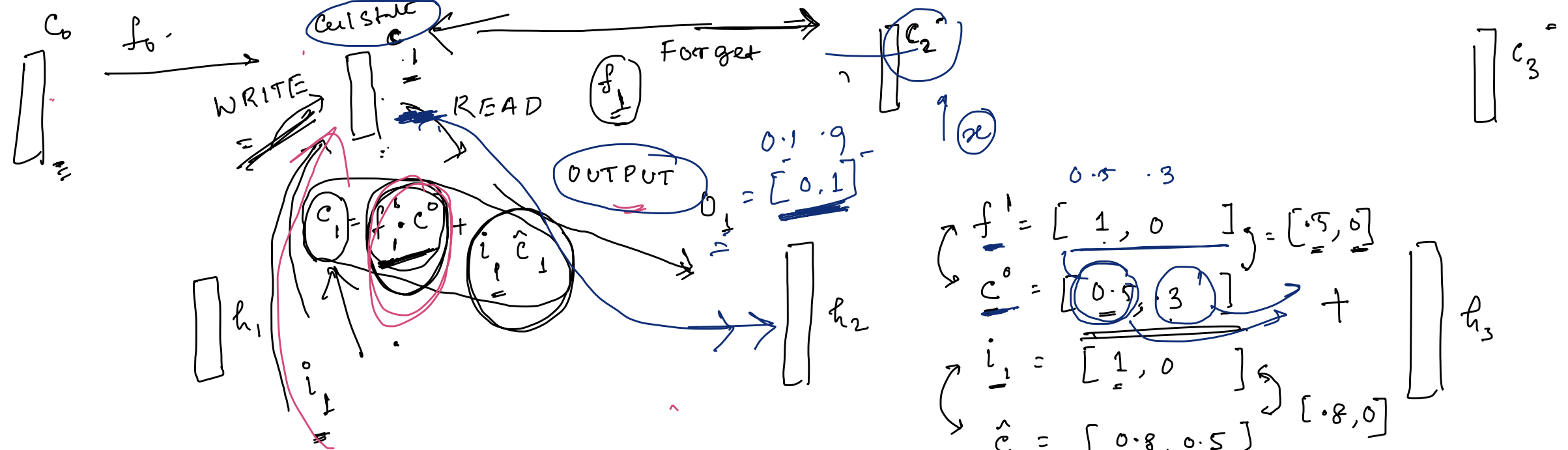
$$h^{(t)} = o^{(t)} \odot \tanh(c^{(t)})$$

f
 i
 o
 W_h
 W_c

All these are vectors of same length n

 $h = o \odot c$

Gates are applied using element-wise (or Hadamard) product: $\odot$



$$\text{Cell content } (\hat{c}) = \sigma(W_c h^0 + W_c e_1 + b)$$

$$i_1 = [0.2, 0.3]$$

$$f^1 = [1, 0] \quad g = [0.5, 0]$$

$$c^0 = [0.5, 3]$$

$$i_1 = [1, 0]$$

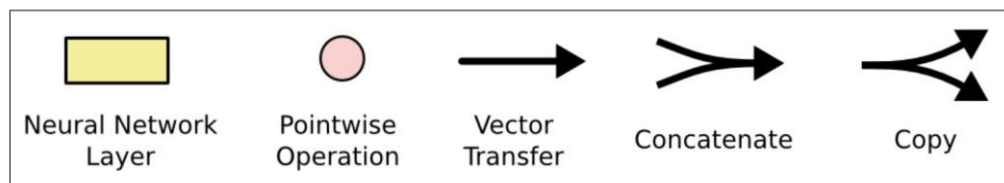
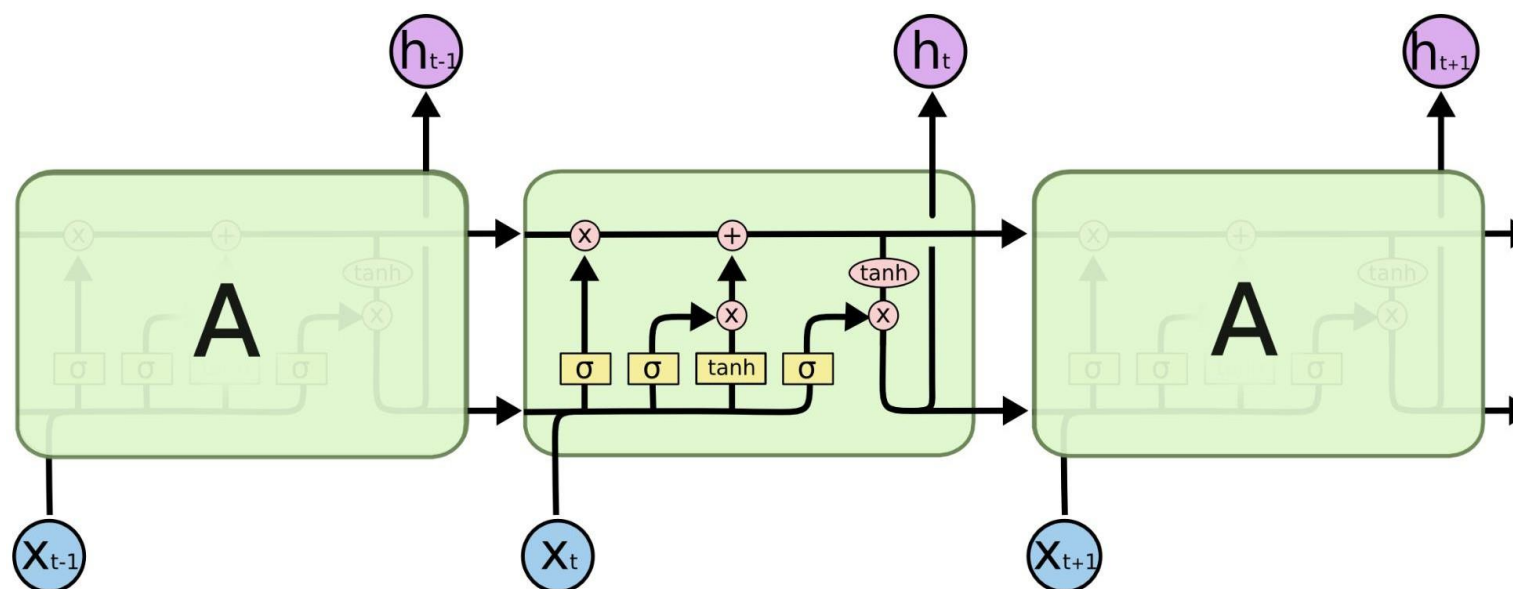
$$\hat{c} = [0.8, 0.5]$$

$$c_1 = [1.3, 0]$$

$$c_1 \cdot 0_1 = [0.8]$$

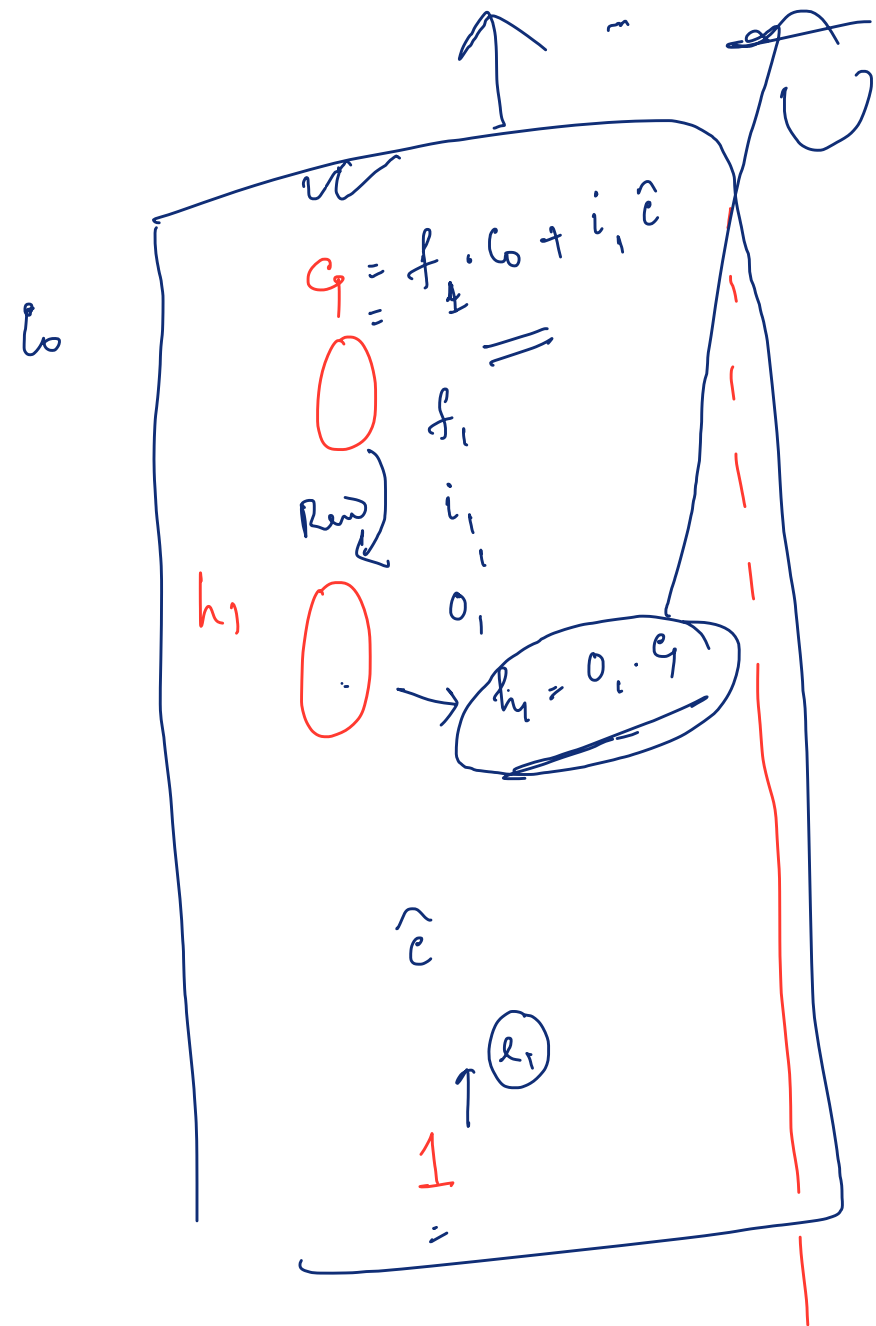
Long Short-Term Memory (LSTM)

You can think of the LSTM equations visually like this:



Source: <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>





$c_2 = f_2 \cdot g + i_2 \cdot \hat{c}$

$\text{Re } c_2$

$h_2 = 0_2 \cdot c_2$

i_2

0_2

$\hat{c}$

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

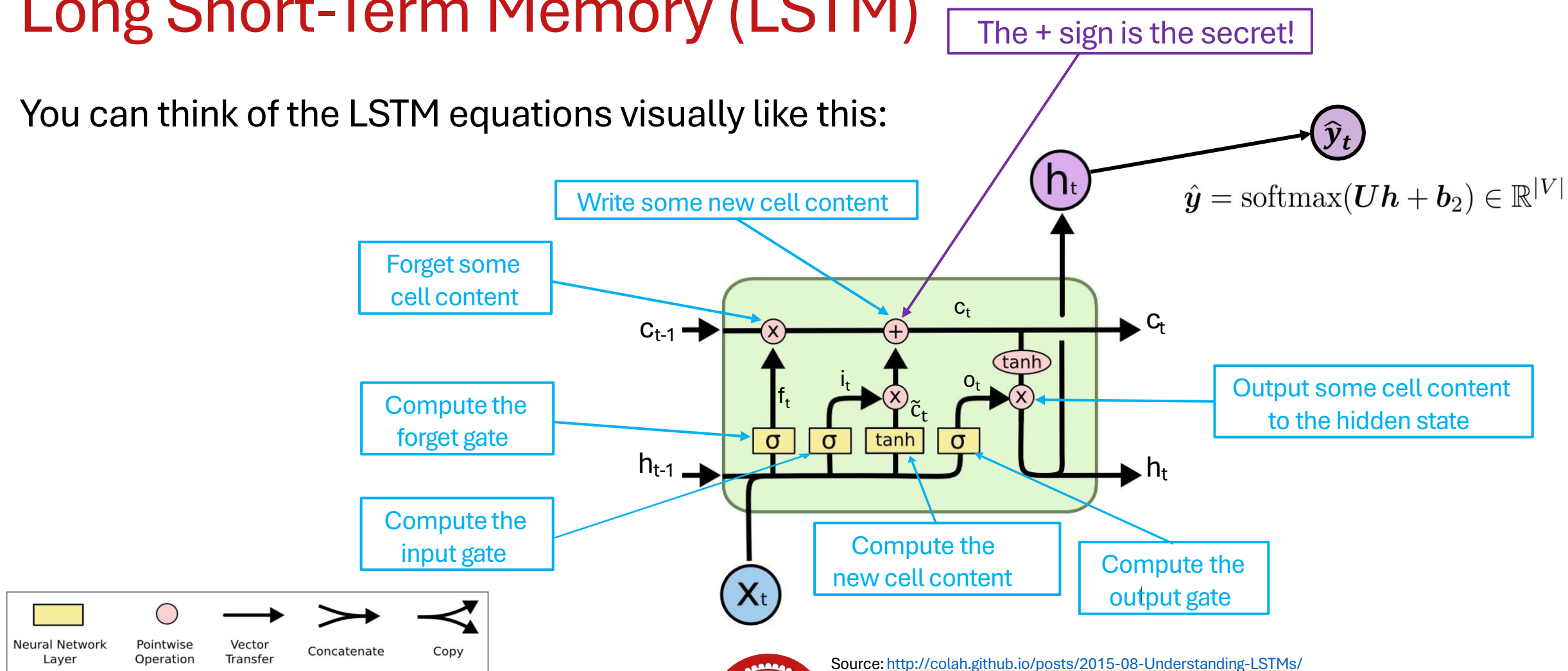
917

918

$$

Long Short-Term Memory (LSTM)

You can think of the LSTM equations visually like this:



How Does LSTM Solve Vanishing Gradients?

- The LSTM architecture makes it **much easier** for an RNN to **preserve information over many timesteps**
 - For example, if the forget gate is set to 1 for a cell dimension and the input gate set to 0, then the information of that cell is preserved indefinitely.
 - In contrast, it's harder for a vanilla RNN to learn a recurrent weight matrix W_h that preserves info in the hidden state
 - In practice, you get about 100 timesteps rather than about 7
- **LSTM doesn't guarantee that there is no vanishing/exploding gradient**, but it does provide an easier way for the model to learn long-distance dependencies.
- There are also alternative ways of creating more direct and linear pass-through connections in models for long distance dependencies.



Gated Recurrent Units (GRUs)

- Proposed by Cho et al. in 2014 as a **simpler alternative to the LSTM**.
- On each timestep t , we have input $x^{(t)}$ and hidden state $h^{(t)}$ (**no cell state**).

Update gate: controls what parts of hidden state are updated vs preserved

Reset gate: controls what parts of previous hidden state are used to compute new content

New hidden state content: reset gate selects useful parts of prev hidden state. Use this and current input to compute new hidden content.

Hidden state: update gate simultaneously controls what is kept from previous hidden state, and what is updated to new hidden state content

How does this solve vanishing gradient?

Like LSTM, GRU makes it easier to retain info long-term (e.g. by setting update gate to 0)

$$u^{(t)} = \sigma(W_u h^{(t-1)} + U_u x^{(t)} + b_u)$$

$$r^{(t)} = \sigma(W_r h^{(t-1)} + U_r x^{(t)} + b_r)$$

$$\tilde{h}^{(t)} = \tanh(W_h (r^{(t)} \circ h^{(t-1)}) + U_h x^{(t)} + b_h)$$

$$h^{(t)} = (1 - u^{(t)}) \circ h^{(t-1)} + u^{(t)} \circ \tilde{h}^{(t)}$$

"Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation", Cho et al. 2014, <https://arxiv.org/pdf/1406.1078v3.pdf>



LSTM vs GRU

- Researchers have proposed many gated RNN variants, but LSTM and GRU are the most widely-used.
- The biggest difference is that GRU is quicker to compute and has fewer parameters.
- There is no conclusive evidence that one consistently performs better than the other.
- LSTM is a good default choice (especially if your data has particularly long dependencies, or you have lots of training data).
- **Rule of thumb:** start with LSTM, but switch to GRU if you want something more efficient.

